

Pointer * Introduction

* what mean Pointer and why?
 ↳ Variable store value →
 ↳ Value is Address

* what type Pointer?

- Pointer to object
 - ↳ Pointer to int
 - ↳ Pointer to char
 - ↳ Pointer to float
 - ↳ Pointer to double
 - ↳ Pointer to struct
 - ↳ Pointer to string
 - ↳ Pointer to array
 - ↳ Pointer to Pointer

Pointer to function

* what is Pointer size?

- ↳ Pointer variable have fixed size
- ↳ Fixed size → same size system bus
- ↳ system bus ⇒ int size



* where the Pointer is store?

- ↳ Pointer is variable
 - local
 - global
 - static local
- ↳ Pointer can be store in (stack / data segment)

Pointer to object

Pointee step *Pointer Name = Init Address; [Definition]

Pointee step * Pointer Name; [Declaration]

* Pointer int

```

int x = 5;
int y = 10;
short int z = 30;

int *ptr1 = &x;
short int *ptr2;
ptr2 = &z;
    
```

* Pointer to char (&) Address operator

```

char y = 30; char z = 200;
char *ptr1; char *ptr2 = &z;
ptr1 = &y;
    
```

* Pointer to struct

```

typedef struct {
    int ID;
    int salary;
} Info_t;

Info_t obj1 = {909;
Info_t *ptr1 = &obj1;
Info_t obj2 = {909;
Info_t *ptr2;
ptr2 = &obj2;
    
```

Definition | Declaration

* Pointer to string

```

char name[] = "NTI";
char *ptr = name;
    
```

Pointer to Array

```

int arr[3] = {0};
int *ptr = arr;
    
```

Array of Pointer

```

int *ptr[3];
int *ptr1;
int *ptr2;
    
```

* How store it in Memory?

```

char x = 20;
char *ptr = &x;
    
```

* How Read / Modify by Pointer?

→ dereferences → *

Create New Pointer

* Definition
 * Declaration

```

*ptr = 70;
ptr = *ptr;
    
```

Operation with Pointer

```

char x = 30;
char y = 60;
char z = 70;
char A = 100;

char *ptr;
    
```

```

ptr = &y;
*ptr = *ptr + 2;
ptr = 0x01 + (2 * 1)
ptr = 0x03
    
```

No of step

```

*ptr = ptr - 3;
= 0x03 - (3 * 1) = 0x00 → 0
    
```

* ① Pointer Can Add or sub Some steps

* ② For know the step size from Pointee step

* Pointer to Function

Dec

```
returnType (*PointerName)(inputs);
PointerName = FunctionName;
ex:
int Add(int x, int y);
int (*AddPF)(int, int);
Add_PF = Add;
```

Definition

```
returnType (*PointerName)(input) =
FunctionName;
int getTemp(void);
int (*getTempPF)(void) = getTemp;
```

How Access Function by Pointer

→ Use the Pointer Name

ex:

```
R = Add_PF(5, 8);
temp = getTemp_PF();
```

* Call by Reference

Pass Pointer to Object

```
→ int Add(int *x, int *y);
→ void swap(int *i, int *j);
```

* Passing Pointer to Function
void CallBack (void (*PF)(void))
Pointer to Function

* How Store Memory?

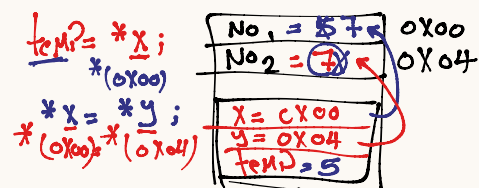
* Function Implementation store in ROM (.text)

* Pointer to Function store in Stack / data segment

* Pointer to Function Point on Address in ROM

```
void swap(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

```
int *x = &No1;
int *y = &No2;
x → Address of No1
*y → Value of No1
*y → Value of No2
```



* Function Argument

```
void Function(int *x, int *y)
```

* You can take pointer as Argument;

* You can return pointer but will be provided logic error.

ex

```
int * Add(int x, int y)
{
    int temp = x + y;
    return &temp;
}
```

Dangling Pointer

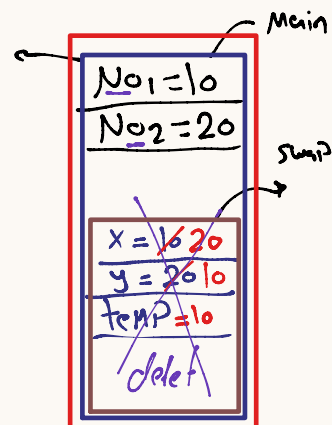
swap

```
void swap(int x, int y)
```

```
{
    temp = x;
    x = y;
    y = temp;
}
```

```
int main()
```

```
{
    int No1 = 10;
    int No2 = 20;
    swap(No1, No2);
}
```

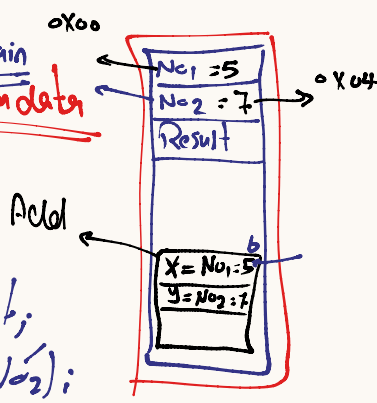


* Calling type

→ Call by value → Copy From data

```
int Add(int x, int y)
{
    return x + y;
}
```

```
int main()
{
    int No1, No2, Result;
    Result = Add(No1, No2);
}
```



Calling type

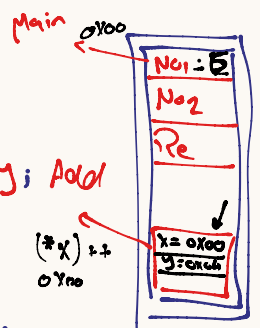
→ Call by reference

```
int Add(int *x, int *y)
```

```
{
    int temp = *x + *y;
    Add(*x, *y);
}
```

```
return temp;
```

```
Main → No1 = 5, No2 = 10
Res = Add(&No1, &No2);
```

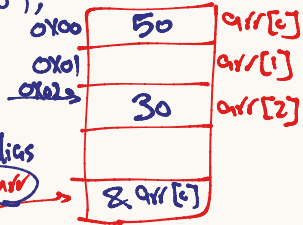


* Pointer & Array Relation

char arr[3] = '09';

arr[0] = 50;

arr[2] = 30;



* array Name is Address first element

arr[1] $\rightarrow$ $*(arr + 1) \rightarrow (0x00 + 1) \rightarrow (0x01)$

arr[Index] $\Rightarrow$ $*(arr + Index)$ step

$\&arr[Index]$ $\Rightarrow$ ~~$\&$~~ $*(arr + Index)$

Address $\rightarrow$ (arr + Index)
 $(0x00 + 3)$
 $(0x03) \rightarrow$

de reference